

MACHINE LEARNING

Comprehensive Study Notes

Covering All 5 Units of the Syllabus

Unit 1: Introduction to ML (5 hrs)

Unit 2: Preprocessing (6 hrs)

Unit 3: Regression (12 hrs)

Unit 4: Classification (15 hrs)

Unit 5: Clustering (7 hrs)

UNIT 1: Introduction to Machine Learning

1.1 Basic Definitions and Concepts

Machine Learning (ML) is a subfield of Artificial Intelligence (AI) that enables computers to learn from experience (data) without being explicitly programmed for every task. The term was coined by Arthur Samuel (1959). Tom Mitchell (1997) provided a formal definition:

Mitchell's Definition

A computer program is said to learn from experience E , with respect to task T and performance measure P , if its performance on T improves with experience E .

Key concepts to understand:

- **Model:** A mathematical representation learned from data.
- **Training:** The process of fitting a model to data.
- **Inference/Prediction:** Using a trained model on new data.
- **Feature:** An individual measurable property or characteristic of a data point (also called input variable or attribute).
- **Label/Target:** The output variable the model tries to predict.
- **Dataset:** A collection of data samples used for training or testing.
- **Generalization:** Ability of a model to perform well on unseen data.

1.2 Key Elements of Machine Learning

Three fundamental components form the basis of any ML system:

- **Data:** Raw information used for training. Quality and quantity of data are critical. Includes training data, validation data, and test data.
- **Model (Algorithm):** Mathematical framework that maps inputs to outputs. Examples: linear regression, decision trees, neural networks.
- **Optimization / Learning:** The process by which the model updates its parameters to minimize errors. Uses techniques like Gradient Descent.

Additional elements:

- **Loss Function:** Measures how wrong the model's predictions are (e.g., Mean Squared Error for regression, Cross-Entropy for classification).
- **Hyperparameters:** Settings chosen before training (e.g., learning rate, number of trees). Unlike parameters, they are not learned from data.
- **Evaluation Metrics:** Measures used to assess model quality (e.g., accuracy, precision, recall, R-squared).

1.3 Types of Machine Learning

1.3.1 Supervised Learning

The algorithm learns from labeled training data — each input has a corresponding output. The goal is to learn a mapping function from inputs to outputs.

- Regression: Predicts a continuous output. Example: predicting house prices.
- Classification: Predicts a discrete class label. Example: email spam detection.

Examples of supervised algorithms: Linear Regression, Logistic Regression, Decision Trees, SVM, Neural Networks.

1.3.2 Unsupervised Learning

The algorithm is given data without labels. The goal is to find hidden patterns, structures, or groupings in the data.

- Clustering: Groups similar data points together. Example: customer segmentation. (K-Means, Hierarchical Clustering)
- Dimensionality Reduction: Reduces the number of features while preserving information. Example: PCA (Principal Component Analysis).
- Association: Finds rules in data. Example: market basket analysis.

Examples: K-Means, DBSCAN, PCA, Autoencoders.

1.3.3 Introduction to Reinforcement Learning (RL)

An agent learns to take actions in an environment to maximize cumulative reward. No labeled data; learning happens through trial and error.

- Agent: The learner or decision-maker.
- Environment: What the agent interacts with.
- State: Current situation of the agent.
- Action: Choices the agent can make.
- Reward: Feedback signal after taking an action.
- Policy: Strategy the agent uses to decide actions.

RL Goal

Maximize: $E[\sum \gamma^t * r_t]$ where γ is the discount factor, r_t is reward at time t

Examples: AlphaGo (game playing), robotic control, recommendation systems.

1.4 Applications of Machine Learning

- Healthcare: Disease prediction, medical imaging, drug discovery.
- Finance: Fraud detection, stock price prediction, credit scoring.
- Natural Language Processing (NLP): Machine translation, sentiment analysis, chatbots.

- Computer Vision: Object detection, facial recognition, self-driving cars.
- Recommendation Systems: Netflix movie suggestions, Amazon product recommendations.
- Speech Recognition: Siri, Google Assistant, Alexa.
- Manufacturing: Predictive maintenance, quality control.

NOTE: ML differs from traditional programming. In traditional programming: Input + Rules = Output. In ML: Input + Output = Rules (the model learns the rules from data).

UNIT 2: Data Preprocessing

Raw data is often incomplete, inconsistent, and noisy. Preprocessing transforms raw data into a format suitable for ML algorithms. This step significantly impacts model performance.

2.1 Feature Scaling

Feature scaling ensures all features contribute equally to the model. Without scaling, features with large numeric ranges dominate distance-based or gradient-based algorithms.

2.1.1 Min-Max Normalization (Rescaling)

Scales features to a fixed range, typically [0, 1].

Min-Max Formula

$$X_{\text{scaled}} = (X - X_{\text{min}}) / (X_{\text{max}} - X_{\text{min}})$$

Use when: distribution is not Gaussian, when a bounded range is needed (e.g., neural network inputs).

Drawback: Sensitive to outliers — a single extreme value can compress all other values into a very narrow range.

2.1.2 Standardization (Z-score Normalization)

Centers data around mean 0 with standard deviation 1.

Z-score Formula

$$X_{\text{scaled}} = (X - \text{mean}) / \text{std_dev}$$

Use when: algorithm assumes Gaussian distribution (e.g., Linear Regression, Logistic Regression, SVM). Less affected by outliers than Min-Max.

2.1.3 Robust Scaling

Uses median and interquartile range (IQR). Robust to outliers.

Robust Scaler

$$X_{\text{scaled}} = (X - \text{median}) / \text{IQR} \quad \text{where } \text{IQR} = Q3 - Q1$$

2.2 Feature Selection Methods

Feature selection reduces the number of input features, reducing overfitting, improving accuracy, and reducing training time. There are three main types:

2.2.1 Filter Methods

Select features based on statistical scores, independent of the learning algorithm.

- Correlation Coefficient: Measures linear relationship between feature and target. Remove highly correlated features.
- Chi-Square Test: Used for categorical features and categorical target.
- ANOVA F-Test: Measures variance between groups relative to variance within groups.
- Mutual Information: Measures the dependence between two variables (works with any distribution).

2.2.2 Wrapper Methods

Use a ML model to evaluate subsets of features. Computationally expensive but often more accurate.

- Forward Selection: Start with no features, add one feature at a time that best improves model performance.
- Backward Elimination: Start with all features, remove the least important feature iteratively.
- Recursive Feature Elimination (RFE): Trains a model, ranks features by importance, removes least important, repeats.

2.2.3 Embedded Methods

Feature selection occurs as part of the model training process.

- LASSO Regression (L1 Regularization): Shrinks some coefficients exactly to zero, effectively removing those features.
- Tree-based methods: Decision trees and Random Forests naturally rank feature importance.

2.3 Dimensionality Reduction

Reduces the number of features by transforming data into a lower-dimensional space. Addresses the 'Curse of Dimensionality' — as dimensions increase, data becomes sparse and distances lose meaning.

2.3.1 Principal Component Analysis (PCA)

PCA is the most widely used linear dimensionality reduction technique. It projects data onto directions of maximum variance (called Principal Components).

Steps in PCA:

1. Standardize the data (zero mean, unit variance).
2. Compute the Covariance Matrix: $C = \frac{1}{(n-1)} * X^T * X$
3. Compute Eigenvectors and Eigenvalues of the covariance matrix.
4. Sort eigenvectors by descending eigenvalues (highest variance first).
5. Select top k eigenvectors as the new feature space.
6. Transform the original data: $Z = X * W_k$ (where W_k contains top-k eigenvectors)

Variance Explained

Explained Variance Ratio = $\lambda_i / \sum(\lambda)$
where λ_i is the i -th eigenvalue

- Each principal component (PC) is a linear combination of original features.
- PCs are orthogonal (uncorrelated) to each other.
- The first PC captures the most variance; subsequent PCs capture decreasing amounts.
- Choose k components that explain a sufficient proportion of variance (e.g., 95%).

NOTE: PCA is unsupervised — it does not consider class labels. For supervised dimensionality reduction, use Linear Discriminant Analysis (LDA).

2.3.2 Other Dimensionality Reduction Techniques

- Linear Discriminant Analysis (LDA): Supervised; maximizes class separability.
- t-SNE (t-Distributed Stochastic Neighbor Embedding): Non-linear; used for visualization of high-dimensional data.
- Autoencoders: Neural network-based; learns compressed representations.

2.4 Handling Missing Data

- Deletion: Remove rows/columns with missing values (only if data loss is acceptable).
- Mean/Median/Mode Imputation: Replace missing values with the column's mean, median, or mode.
- Predictive Imputation: Use a model to predict missing values from other features.
- Indicator Variable: Add a binary column indicating whether the value was missing.

2.5 Encoding Categorical Variables

- Label Encoding: Assigns integer values to categories (e.g., Red=0, Green=1, Blue=2). Suitable for ordinal categories.
- One-Hot Encoding: Creates binary columns for each category. Suitable for nominal categories. Avoids false ordinality.
- Target Encoding: Replaces category with mean target value. Risk of data leakage if not done carefully.

UNIT 3: Regression

Regression is a supervised learning task where the output is a continuous numerical value. It models the relationship between one or more independent variables (features) and a dependent variable (target).

3.1 Linear Regression with One Variable (Simple Linear Regression)

Models the relationship between one input feature x and one output y as a straight line.

Model	$h(x) = \theta_0 + \theta_1 * x$
--------------	----------------------------------

Where: θ_0 = bias (y-intercept), θ_1 = weight (slope), $h(x)$ = predicted output (hypothesis).

3.1.1 Cost Function

Measures the difference between predicted and actual values. The most common is Mean Squared Error (MSE):

MSE Cost Function	$J(\theta) = (1 / 2m) * \sum_{i=1}^m (h(x^{(i)}) - y^{(i)})^2$
--------------------------	--

Where m = number of training examples. The factor $1/2$ simplifies the derivative calculation.

3.1.2 Ordinary Least Squares (OLS) — Normal Equation

Analytical solution to find optimal parameters in one shot (no iterations):

Normal Equation	$\theta = (X^T * X)^{-1} * X^T * y$
------------------------	-------------------------------------

Suitable for small datasets. Computationally expensive ($O(n^3)$) for large feature sets ($n > 10,000$).

3.2 Linear Regression with Multiple Variables

Extends simple linear regression to multiple input features ($x_1, x_2, \dots, x_n$):

Multiple LR Model	$h(x) = \theta_0 + \theta_1 * x_1 + \theta_2 * x_2 + \dots + \theta_n * x_n = \theta^T * x$
--------------------------	---

Feature scaling is critical for multiple linear regression to ensure all features contribute proportionally and gradient descent converges faster.

3.3 Gradient Descent

An iterative optimization algorithm to minimize the cost function by updating parameters in the direction of the negative gradient.

Update Rule	$\theta_j := \theta_j - \alpha * (dJ/d_{\theta_j})$
--------------------	---

Gradient (Simple LR)

$$\frac{dJ}{d_{\theta_j}} = (1/m) * \sum_{i=1}^m (h(x^{(i)}) - y^{(i)}) * x_j^{(i)}$$

Where alpha = learning rate (controls step size).

3.3.1 Types of Gradient Descent

- Batch Gradient Descent: Uses all training examples per update. Stable but slow for large datasets.
- Stochastic Gradient Descent (SGD): Uses one training example per update. Fast but noisy convergence.
- Mini-Batch Gradient Descent: Uses a small batch (e.g., 32 or 64 examples). Balance of speed and stability. Most commonly used in practice.

3.3.2 Choosing the Learning Rate

- Too large: Overshoots minimum, diverges.
- Too small: Converges very slowly.
- Adaptive methods (Adam, RMSProp) automatically adjust learning rate during training.

NOTE: Convergence: Plot $J(\theta)$ vs. number of iterations. If cost is decreasing, algorithm is working. If oscillating or increasing, reduce alpha.

3.4 Overfitting and Underfitting

3.4.1 Underfitting (High Bias)

The model is too simple to capture the underlying pattern in the data. Performs poorly on both training and test data.

- Cause: Model complexity is too low, too few features, insufficient training.
- Fix: Increase model complexity, add more features, reduce regularization.

3.4.2 Overfitting (High Variance)

The model learns the training data too well, including its noise and random fluctuations. Performs very well on training data but poorly on unseen test data.

- Cause: Model too complex, too many features, too little data, training for too long.
- Fix: Regularization, more training data, feature selection, early stopping, dropout (for neural networks).

3.4.3 The Bias-Variance Tradeoff

- Bias: Error from wrong assumptions in the model. High bias = underfitting.

- Variance: Error from sensitivity to fluctuations in training data. High variance = overfitting.

Total Error

$$\text{Total Error} = \text{Bias}^2 + \text{Variance} + \text{Irreducible Noise}$$

Goal: Find the sweet spot with low bias AND low variance.

3.5 Regularization

Regularization adds a penalty term to the cost function to discourage large parameter values, reducing overfitting.

3.5.1 Ridge Regression (L2 Regularization)

Ridge Cost

$$J(\theta) = \text{MSE} + \lambda \cdot \sum (\theta_j^2)$$

L2 penalty shrinks all coefficients toward zero but never exactly to zero. Useful when all features contribute somewhat. λ controls the strength of regularization.

3.5.2 LASSO Regression (L1 Regularization)

LASSO Cost

$$J(\theta) = \text{MSE} + \lambda \cdot \sum (|\theta_j|)$$

L1 penalty can shrink some coefficients exactly to zero, performing automatic feature selection. Useful when many features are irrelevant.

3.5.3 Elastic Net

Combination of L1 and L2 regularization. Used when there are many correlated features.

Elastic Net Cost

$$J(\theta) = \text{MSE} + \lambda_1 \cdot \sum (|\theta_j|) + \lambda_2 \cdot \sum (\theta_j^2)$$

3.6 Regression Evaluation Metrics

- Mean Absolute Error (MAE): Average of absolute differences. Robust to outliers. $\text{MAE} = (1/m) \cdot \sum |y_{\text{pred}} - y_{\text{actual}}|$
- Mean Squared Error (MSE): Average of squared differences. Penalizes large errors more. $\text{MSE} = (1/m) \cdot \sum (y_{\text{pred}} - y_{\text{actual}})^2$
- Root Mean Squared Error (RMSE): Square root of MSE. Same unit as target variable. More interpretable than MSE.
- R-squared (R^2 , Coefficient of Determination): Proportion of variance in the target explained by the model. $R^2 = 1 - \text{SS}_{\text{res}}/\text{SS}_{\text{tot}}$. Range: 0 to 1 (higher is better). $R^2=1$ = perfect fit; $R^2=0$ = model no better than predicting the mean.
- Adjusted R-squared: Adjusted for number of features. Penalizes adding irrelevant features.

UNIT 4: Classification

Classification is a supervised learning task where the output is a discrete class label. The algorithm learns a boundary to separate different classes.

4.1 Decision Trees

A tree-shaped model that makes decisions by asking a series of questions about the features. Easy to interpret and visualize.

4.1.1 Structure

- Root Node: The topmost node; represents the entire dataset; split using the best feature.
- Internal Nodes: Represent features/tests. Each branch is an outcome.
- Leaf Nodes: Terminal nodes that represent class labels (predictions).

4.1.2 Splitting Criteria

Measures used to decide the best split at each node:

- Gini Impurity (used in CART): $Gini = 1 - \sum(p_i^2)$. Measures probability of misclassification. Range: 0 (pure) to 0.5 (maximum impurity for binary).
- Entropy and Information Gain (used in ID3, C4.5): Entropy = $-\sum(p_i * \log_2(p_i))$. Information Gain = Entropy(parent) - weighted_avg(Entropy(children)). Higher gain = better split.
- Variance Reduction: Used for regression trees.

4.1.3 Pruning

- Pre-pruning: Stop growing the tree early (e.g., max depth, min samples per leaf).
- Post-pruning: Grow full tree then remove branches that don't improve generalization (e.g., reduced error pruning).

NOTE: Decision trees are prone to overfitting if not pruned. Ensemble methods like Random Forests combine many trees to improve accuracy and reduce overfitting.

4.2 Naive Bayes Classifier

A probabilistic classifier based on Bayes' Theorem with the 'naive' assumption that all features are conditionally independent given the class label.

Bayes' Theorem

$$P(C|X) = P(X|C) * P(C) / P(X)$$

$P(C|X)$ = Posterior probability of class C given features X. $P(X|C)$ = Likelihood. $P(C)$ = Prior probability. $P(X)$ = Evidence (normalizing constant).

The classification rule:

Naive Bayes Rule

$$y_{\text{pred}} = \underset{C}{\operatorname{argmax}} [P(C) * \prod_i P(x_i|C)]$$

4.2.1 Types of Naive Bayes

- Gaussian NB: Assumes features follow Gaussian (normal) distribution. Used for continuous features.
- Multinomial NB: Used for discrete counts (e.g., word counts in text classification).
- Bernoulli NB: Used when features are binary (0 or 1).

Advantages: Fast, works well with small data, handles high dimensions. Disadvantage: The independence assumption is often violated in real data.

4.3 Logistic Regression

Despite the name, logistic regression is a classification algorithm. It models the probability of a binary outcome using the sigmoid (logistic) function.

Sigmoid Function

$$\sigma(z) = 1 / (1 + e^{(-z)}) \quad \text{where } z = \theta^T * x$$

Output: $P(y=1|x) = \sigma(\theta^T * x)$. Range: (0, 1). Decision boundary: Predict class 1 if $P > 0.5$, else class 0.

4.3.1 Cost Function

Uses Log Loss (Binary Cross-Entropy) instead of MSE (MSE creates non-convex surface):

Log Loss

$$J = -(1/m) * \sum [y * \log(h) + (1-y) * \log(1-h)]$$

4.3.2 Multiclass Logistic Regression

- One-vs-Rest (OvR): Train one classifier per class against all others.
- Softmax Regression: Generalizes logistic regression to multiple classes directly.

Can use L1 or L2 regularization to prevent overfitting.

4.4 K-Nearest Neighbor (KNN) Classifier

A non-parametric, instance-based (lazy) learning algorithm. No explicit training phase — the model simply memorizes the training data.

4.4.1 Algorithm

7. Choose k (number of neighbors).
8. For a new test point, compute distance to all training points.
9. Find the k training points with smallest distance.

10. Assign the class that appears most frequently among k neighbors (majority vote).

4.4.2 Distance Metrics

Euclidean Distance	$d = \sqrt{\sum (x_i - y_i)^2}$
Manhattan Distance	$d = \sum x_i - y_i $
Minkowski Distance	$d = (\sum x_i - y_i ^p)^{1/p}$ (p=2: Euclidean, p=1: Manhattan)

- Choosing k: Small k = noisy boundary (overfitting). Large k = smooth boundary (underfitting). Use cross-validation to find optimal k.
- Feature scaling is essential for KNN — otherwise features with large ranges dominate distance.

*NOTE: KNN is computationally expensive at prediction time: $O(n * d)$ per prediction where n = training size, d = dimensions. Not suitable for very large datasets without optimization.*

4.5 Perceptron and Neural Networks

4.5.1 Perceptron (Single Neuron)

The simplest neural model, inspired by biological neurons. Takes a weighted sum of inputs and applies a step function.

Perceptron Output	$y = f(w^T * x + b)$ where f is a step/activation function
--------------------------	--

The Perceptron Learning Rule updates weights when a misclassification occurs:

Weight Update	$w := w + \alpha * (y - y_{pred}) * x$
----------------------	--

Limitation: Can only learn linearly separable patterns (XOR problem cannot be solved).

4.5.2 Multilayer Perceptron (MLP) and Neural Networks

A MLP has multiple layers: an input layer, one or more hidden layers, and an output layer.

- Activation Functions: Introduce non-linearity, allowing the network to learn complex patterns.
 - Sigmoid: Output (0,1). Used in output layer for binary classification.
 - Tanh: Output (-1,1). Better than sigmoid for hidden layers.
 - ReLU (Rectified Linear Unit): $f(x) = \max(0, x)$. Most common for hidden layers; avoids vanishing gradient.
 - Softmax: Output layer for multi-class classification.
- Forward Propagation: Input flows through layers, computing activations layer by layer.

- Backpropagation: Algorithm to compute gradients of the loss with respect to all weights, using the chain rule of calculus. Gradients flow backward through the network.
- Weight Update: Parameters are updated using gradient descent after backpropagation.

4.5.3 Key Hyperparameters

- Number of hidden layers and neurons per layer.
- Learning rate and optimizer (SGD, Adam, RMSProp).
- Batch size and number of epochs.
- Regularization: L2 (weight decay), Dropout (randomly deactivates neurons during training).

4.6 Support Vector Machine (SVM)

SVM is a powerful classification algorithm that finds the optimal hyperplane that maximally separates classes by the largest margin.

4.6.1 Maximum Margin Classifier

The decision boundary (hyperplane) is: $w^T \cdot x + b = 0$. Support Vectors are the data points closest to the hyperplane. Margin = $2 / ||w||$. SVM maximizes margin by minimizing $||w||$.

SVM Optimization

Minimize: $(1/2) ||w||^2$ Subject to: $y_i (w^T x_i + b) \geq 1$ for all i

4.6.2 Soft Margin SVM

Real data is not perfectly separable. Soft margin SVM allows some misclassifications using slack variables (ξ_i):

Soft Margin

Minimize: $(1/2) ||w||^2 + C * \sum(\xi_i)$

C is the regularization parameter: Large C = fewer misclassifications (hard margin, risk of overfitting). Small C = allows more misclassifications (wider margin, better generalization).

4.6.3 Kernel Trick

SVM can solve non-linearly separable problems by mapping data to higher dimensions using kernel functions:

- Linear Kernel: $K(x,z) = x^T \cdot z$. For linearly separable data.
- Polynomial Kernel: $K(x,z) = (x^T \cdot z + c)^d$.
- Radial Basis Function (RBF/Gaussian) Kernel: $K(x,z) = \exp(-\gamma * ||x-z||^2)$. Most popular for non-linear data.

The kernel trick computes the dot product in the higher-dimensional space without explicitly mapping to it — computationally efficient.

4.7 Classification Evaluation Metrics

4.7.1 Confusion Matrix

A 2x2 matrix (for binary classification) showing True Positives (TP), True Negatives (TN), False Positives (FP — Type I Error), False Negatives (FN — Type II Error).

4.7.2 Key Metrics

Accuracy	$\text{Accuracy} = (\text{TP} + \text{TN}) / (\text{TP} + \text{TN} + \text{FP} + \text{FN})$
Precision	$\text{Precision} = \text{TP} / (\text{TP} + \text{FP})$ — of all predicted positives, how many are correct?
Recall (Sensitivity)	$\text{Recall} = \text{TP} / (\text{TP} + \text{FN})$ — of all actual positives, how many did we catch?
F1-Score	$\text{F1} = 2 * (\text{Precision} * \text{Recall}) / (\text{Precision} + \text{Recall})$ — harmonic mean
Specificity	$\text{Specificity} = \text{TN} / (\text{TN} + \text{FP})$

4.7.3 ROC Curve and AUC

- ROC (Receiver Operating Characteristic) Curve: Plots True Positive Rate (Recall) vs. False Positive Rate at various threshold settings.
- AUC (Area Under the ROC Curve): AUC = 1.0 means perfect classifier. AUC = 0.5 means random classifier. Higher AUC = better model.
- Precision-Recall Curve: More informative when classes are imbalanced.

NOTE: Accuracy can be misleading for imbalanced datasets (e.g., 99% class A, 1% class B). Always use Precision, Recall, F1-Score, and AUC for a complete picture.

4.8 Cross-Validation

A technique to estimate model performance on unseen data and reduce overfitting to the train/test split.

- K-Fold Cross-Validation: Split data into k equal folds. Train on k-1 folds, test on remaining fold. Repeat k times. Average the k scores. Typical k = 5 or 10.
- Stratified K-Fold: Maintains class proportions in each fold. Essential for imbalanced datasets.
- Leave-One-Out (LOO): k = n (each sample is a test set once). Expensive for large datasets.

UNIT 5: Clustering

Clustering is an unsupervised learning technique that groups data points such that points within the same group (cluster) are more similar to each other than to points in other groups. There are no labels — the algorithm discovers inherent structure in data.

5.1 Approaches for Clustering

- Partitioning (Centroid-based): Divide data into k non-overlapping subsets. Example: K-Means.
- Hierarchical: Build a tree of clusters. Can be agglomerative (bottom-up) or divisive (top-down).
- Density-based: Form clusters based on regions of high density. Example: DBSCAN. Can find arbitrary-shaped clusters and handle noise/outliers.
- Model-based: Assume data is generated from a mixture of probability distributions. Example: Gaussian Mixture Models (GMM).
- Grid-based: Partition data space into a finite number of cells. Fast for large datasets.

5.2 Distance Metrics in Clustering

Clustering depends heavily on the distance metric used to measure similarity/dissimilarity between data points.

Euclidean Distance	$d(x, y) = \sqrt{\sum (x_i - y_i)^2}$
Manhattan Distance	$d(x, y) = \sum x_i - y_i $
Cosine Similarity	$\cos(x, y) = (x \cdot y) / (x * y)$ Range: -1 to 1; 1 = identical direction
Jaccard Similarity	$J(A, B) = A \cap B / A \cup B $ Used for sets/binary features

- Choice of distance metric greatly affects clustering results.
- Euclidean: Works well for compact, spherical clusters.
- Manhattan: More robust to outliers than Euclidean.
- Cosine: Ideal for text/document similarity (direction matters, not magnitude).

5.3 K-Means Clustering

K-Means is the most popular partitioning clustering algorithm. It partitions n data points into k clusters by minimizing the within-cluster sum of squared distances.

K-Means Objective	Minimize: $\sum_{j=1}^k \sum_{x \in C_j} x - \mu_j ^2$
--------------------------	---

Where μ_j is the centroid (mean) of cluster j .

5.3.1 K-Means Algorithm (Lloyd's Algorithm)

11. Initialize k centroids randomly (or using K-Means++ for better initialization).
12. Assignment Step: Assign each data point to the nearest centroid.
13. Update Step: Recalculate centroids as the mean of all points assigned to each cluster.
14. Repeat steps 2-3 until centroids stop moving (convergence) or max iterations reached.

5.3.2 K-Means++ Initialization

Improves initial centroid placement to avoid poor convergence:

15. Choose first centroid randomly from data points.
16. For each subsequent centroid, choose a data point with probability proportional to its distance squared from the nearest existing centroid.
17. Repeat until k centroids are chosen.

5.3.3 Choosing k — The Elbow Method

Run K-Means for different values of k (e.g., 1 to 10). Plot the Within-Cluster Sum of Squares (WCSS) vs. k . Look for the 'elbow' — the point where adding more clusters gives diminishing returns. The elbow point is the optimal k .

5.3.4 Limitations of K-Means

- Must specify k in advance.
- Sensitive to initial centroid placement (K-Means++ helps).
- Assumes clusters are spherical, equally sized, and have similar density.
- Sensitive to outliers — outliers can skew centroids.
- Cannot handle non-convex cluster shapes.

5.4 Hierarchical Clustering

Builds a hierarchy of clusters without requiring k to be specified in advance. The output is a tree-like diagram called a Dendrogram.

5.4.1 Agglomerative Clustering (Bottom-Up)

The most common hierarchical approach:

18. Start: Each data point is its own cluster (n clusters).
19. At each step, merge the two most similar (closest) clusters.
20. Repeat until all points are in one cluster.

21. Cut the dendrogram at a desired height to get the desired number of clusters.

5.4.2 Linkage Criteria

Defines how distance between clusters is measured:

- Single Linkage (MIN): Distance = minimum distance between any two points in the two clusters. Creates elongated, chain-like clusters. Sensitive to outliers.
- Complete Linkage (MAX): Distance = maximum distance between any two points. Creates compact, spherical clusters.
- Average Linkage: Distance = average of all pairwise distances between clusters. Compromise between single and complete.
- Ward's Linkage: Minimizes the total increase in within-cluster variance when merging. Often produces the most compact and even clusters. Most commonly used.

5.4.3 Dendrogram

A tree diagram that shows the hierarchical merging process. The y-axis represents the distance (dissimilarity) at which clusters were merged. To choose k clusters: draw a horizontal line across the dendrogram. The number of vertical lines it crosses = k.

5.4.4 Divisive Clustering (Top-Down)

Start with all points in one cluster. Recursively split clusters until each point is its own cluster. Less common than agglomerative. Example: DIANA (Divisive ANALysis).

5.5 Clustering Evaluation Metrics

Evaluating clustering is harder than classification since there are no ground truth labels in most cases.

5.5.1 Internal Metrics (No Ground Truth Required)

- Silhouette Score: For each point i , $s(i) = (b(i) - a(i)) / \max(a(i), b(i))$. Where $a(i)$ = average distance to points in same cluster; $b(i)$ = average distance to points in nearest neighboring cluster. Range: -1 to 1. Score close to 1 = well-clustered. Score near 0 = on cluster boundary. Score < 0 = possibly wrong cluster.
- Davies-Bouldin Index: Average ratio of within-cluster scatter to between-cluster separation. Lower is better. DB = 0 means perfect clustering.
- Within-Cluster Sum of Squares (WCSS / Inertia): Total squared distance of points from their cluster centroid. Lower is better. Used in Elbow Method.

5.5.2 External Metrics (Ground Truth Required)

- Rand Index: Fraction of pairs correctly clustered/separated. Range: 0 to 1.

- Adjusted Rand Index (ARI): Corrects for chance. Range: -1 to 1 (1 = perfect, 0 = random).
- Normalized Mutual Information (NMI): Measures mutual dependence between predicted and true cluster assignments.

NOTE: In practice, clustering results should also be evaluated qualitatively — do the discovered clusters make business or domain sense?

5.6 DBSCAN (Density-Based Spatial Clustering of Applications with Noise)

A density-based algorithm that groups together closely packed points and marks outliers as noise.

- Core Point: A point with at least MinPts neighbors within radius epsilon.
- Border Point: Within epsilon of a core point but fewer than MinPts neighbors.
- Noise Point (Outlier): Not a core point and not within epsilon of any core point.

Advantages: Finds arbitrarily shaped clusters. Does not require k. Robust to outliers.

Disadvantages: Struggles with varying density clusters. Sensitive to epsilon and MinPts parameters.

Summary: Quick Reference

Algorithm	Type	Key Parameter	Best For
Linear Regression	Supervised / Regression	Regularization (lambda)	Continuous output prediction
Logistic Regression	Supervised / Classification	Regularization (C)	Binary/multi-class
Decision Tree	Supervised / Both	Max depth, min samples	Interpretable decisions
Naive Bayes	Supervised / Classification	Distribution type	Text, small datasets
KNN	Supervised / Both	k (neighbors)	Simple, small datasets
SVM	Supervised / Both	C, kernel, gamma	High-dim, non-linear
Neural Network	Supervised / Both	Layers, LR, epochs	Complex patterns
K-Means	Unsupervised / Clustering	k (clusters)	Spherical clusters
Hierarchical	Unsupervised / Clustering	Linkage, distance	Unknown k, dendrograms
PCA	Unsupervised / Dim. Red.	k components	Feature reduction